

java.time API

Date/Time : java.time API

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

New Date-Time API in Java 8

Java 8 introduced the New Date-Time API in the `java.time` package to provide a modern, immutable, and thread-safe way of handling dates and times. It replaces the limitations of the older `java.util.Date` and `java.util.Calendar` classes by offering a cleaner design, better readability.

- Immutable and thread-safe design.
- Clear separation of date, time, and timezone classes.
- Easy formatting and parsing of dates.

Why Use the New Date-Time API?

- Date-time objects cannot be modified after creation, making them safe for concurrent applications.
- Provides a clean and intuitive design for working with dates and times.
- Supports formatting, parsing, arithmetic operations, comparisons, and time-zone handling.
- Clearly separates date, time, date-time, and timezone concepts into different classes.
- Uses the international ISO-8601 standard by default.

Main Components of the Java 8 Date-Time API

Local Date-Time Classes

Used when timezone information is not required.

- `LocalDate` -> Represents a date (year, month, day).
- `LocalTime` -> Represents a time (hour, minute, second).
- `LocalDateTime` -> Represents both date and time.

Zoned Date-Time Classes

Used when working with different time zones.

- `ZonedDateTime` -> Represents date and time with timezone.
- `ZoneId` -> Identifies a specific timezone.
- `ZoneOffset` -> Represents the UTC offset from GMT.

Core Classes & Concepts

The java.time API separates temporal data into clear, distinct classes depending on your exact domain needs:

Class	Purpose	Example Output
LocalDate	A date without time or timezone (Year, Month, Day).	2026-10-03
LocalTime	A time without date or timezone (Hour, Minute, Second, Nano).	10:41:22.125
LocalDateTime	A combined date and time without a timezone.	2026-10-03T10:41:22.125
ZonedDateTime	Fully timezone-aware date and time (uses a ZoneId).	2026-10-03T10:41:22.125+05:30[Asia/Kolkata]
Instant	A single instantaneous point on the timeline (always in UTC).	2026-10-03T05:11:22.125Z into table

Sub-Packages of Java Date and Time API

The Java Date and Time API provides specialized sub-packages for advanced date, calendar, formatting, and time-zone operations.

Sub-Package	Purpose	Example / Use Case
<code>java.time.chrono</code>	Supports alternative calendar systems beyond the ISO calendar, such as Hijrah, Minguo, and Thai Buddhist calendars.	Working with dates in different calendar systems.
<code>java.time.temporal</code>	Provides low-level date/time fields, units, and adjustment utilities through classes such as <code>TemporalAdjusters</code> .	Finding the last Friday of a month.
<code>java.time.zone</code>	Provides time-zone rules, historical transitions, and daylight-saving information.	Determining time-zone offsets for a particular date.
<code>java.time.format</code>	Provides classes for parsing and formatting date/time values.	Converting <code>2026-10-03</code> into <code>03-10-2026</code> using <code>DateTimeFormatter</code> .

Context Switching in Java Threads

Context switching is the process where the CPU pauses the execution of one thread and switches to another thread, saving the current thread's execution state and restoring the next thread's state.

In simple words:

Context switching = CPU switching from one thread to another.

- **What Happens During Context Switching?**

Thread A



Executing



CPU

The scheduler decides to run Thread B.

**The system roughly
does:
Thread A**



Save Thread A's state



Context Switch



Load Thread B's state



Thread B



Executing

Later:

Thread B



Save Thread B's state



Context Switch



**Restore Thread A's
state**



Thread A continues

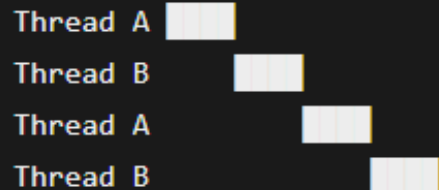
Context Switching vs Parallel Execution

Single CPU Core

With one core:

</> Plain text

Time →



The CPU is actually executing one thread at a time, rapidly switching between them. This gives the appearance of simultaneous execution.

Context Switching Has a Cost

Context switching is not free.

When switching between threads, the system has to save and restore execution state, and the CPU may experience effects such as cache disruption.

Multiple CPU Cores

With multiple cores:

</> Plain text

Core 1 → Thread A

Core 2 → Thread B

Here, Thread A and Thread B can actually execute simultaneously.

This is called **parallelism**.